Résoudre le Job Shop par essaims particulaires

Explication du code du projet `jsp_pso`

Groupe B — Optimisation combinatoire, HEH 2026–2027

Résumé

Ce document explique, fichier par fichier, ce que fait le programme `jsp_pso` : comment il lit une instance, comment une *particule* (une simple liste de nombres) devient un planning, comment l'essaim améliore ces plannings au fil du temps, et comment on l'empêche de stagner. Chaque mécanisme est illustré sur le petit atelier 3×3 des slides du cours, puis mesuré sur des instances de référence (JSPLIB). Le solveur est un PSO *pur* : aucune recherche locale n'y est ajoutée.

Table des matières

1 Le problème : qui passe en premier sur la machine ? 2
- 1.1 L'atelier . . . 2
- 1.2 Même atelier, deux ordres, deux résultats . . . 2
- 1.3 Pourquoi une métaheuristique ? . . . 2

2 Vue d'ensemble du programme 3

3 Lire l'instance (`instance.cpp`) 4

4 Représenter une solution : les clés aléatoires 4

5 Décoder une particule (`schedule.cpp`) 4
- 5.1 Le décodeur semi-actif . . . 4
- 5.2 Le décodeur Giffler-Thompson paramétré (par défaut) . . . 5

6 L'essaim particulaire (`pso.cpp`) 6
- 6.1 Le principe . . . 6
- 6.2 Les réglages retenus . . . 6
- 6.3 La boucle principale . . . 7

7 Empêcher l'essaim de stagner 7
- 7.1 La topologie en anneau . . . 7
- 7.2 Le redémarrage partiel . . . 8
- 7.3 CLPSO : l'apprentissage global . . . 8
- 7.4 Le multi-essaims hétérogène . . . 8

8 Parallélisme et fiabilité 8

9 Résultats 9
- 9.1 Convergence sur l'instance des slides . . . 9
- 9.2 Ce qu'apporte chaque mécanisme . . . 10
- 9.3 Résultats par instance . . . 10
- 9.4 Ce que nous avons appris . . . 10

10 Utilisation 11

1 Le problème : qui passe en premier sur la machine ?

1.1 L'atelier

Un *job shop* est un ensemble de **jobs** à exécuter sur un ensemble de **machines**. Chaque job suit une *recette* fixe : une suite d'opérations, chacune sur une machine précise, pendant une durée précise. Trois contraintes doivent être respectées :

1. **ordre imposé** : les étapes d'un même job se font dans l'ordre de la recette ;
2. **une machine, une opération à la fois** ;
3. **pas d'interruption** : une étape commencée va jusqu'au bout.

Les recettes ne sont pas négociables. **La seule décision, c'est l'ordre** dans lequel chaque machine traite les jobs qui l'utilisent. L'objectif est de minimiser le **makespan**, l'instant où le dernier job se termine.

TABLE 1 – L'exemple des slides : 3 jobs, 3 machines (machine · durée).

	Étape 1	Étape 2	Étape 3
Job 0	M0 · 3	M1 · 2	M2 · 4
Job 1	M1 · 4	M2 · 3	M0 · 2
Job 2	M2 · 2	M0 · 5	M1 · 3

1.2 Même atelier, deux ordres, deux résultats

Plaçons les opérations une par une dans un ordre donné. Chaque opération démarre *au plus tôt* : quand son job est prêt **et** que sa machine est libre.

```
début = max(job prêt, machine libre)    fin = début + durée
```

L'ordre A ci-dessous est la suite $J_1, J_0, J_2, J_2, J_0, J_1, J_1, J_2, J_0$: chaque lettre désigne « la prochaine étape de ce job ». Prenons la cinquième opération placée, J_0 étape 2 sur M1 :

```
J0 a fini son étape 1 à   3
M1 est occupée par J1 jusqu'à 4
début = max(3, 4) = 4   ⇒ J0 attend que la machine se libère
fin   = 4 + 2 = 6
```

1.3 Pourquoi une métaheuristique ?

Pour ce petit atelier, il existe $\frac{9!}{3!\,3!\,3!} = 1\,680$ ordres de placement. Nous les avons tous énumérés : le meilleur donne **11**, le pire **28**, et 384 ordres atteignent l'optimum. Pour 10 jobs et 10 machines (l'exemple `10_10` des slides), le nombre d'ordres monte à $\frac{100!}{(10!)^{10}} \approx 2{,}4 \times 10^{92}$. Il est impossible de tout essayer : il faut une méthode qui *cherche intelligemment*. C'est le rôle de l'optimisation par essaims particulaires.

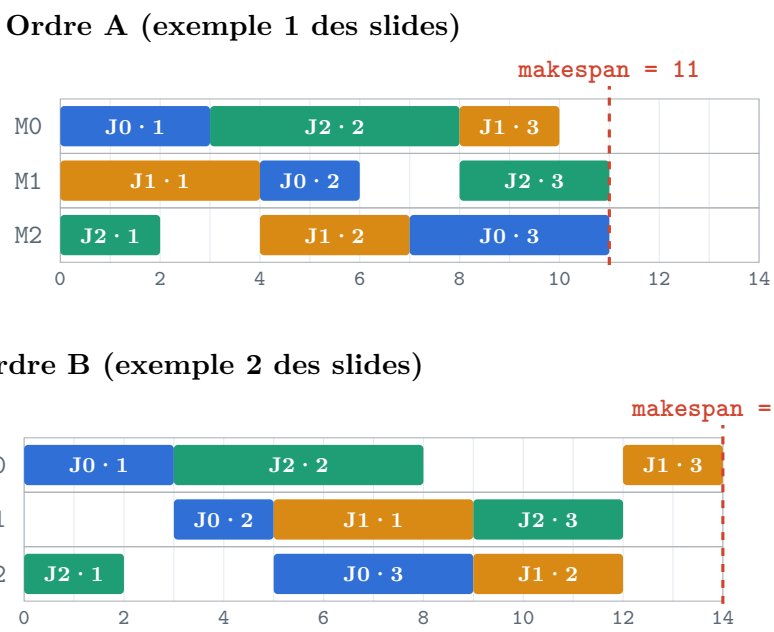


FIGURE 1 – Les mêmes recettes avec deux ordres de placement. Dans l'ordre B, J_0 passe en premier sur M1 : J_1 doit attendre jusqu'à 5 et tout le reste glisse (*effet domino*). Le makespan passe de 11 à 14.

2 Vue d'ensemble du programme

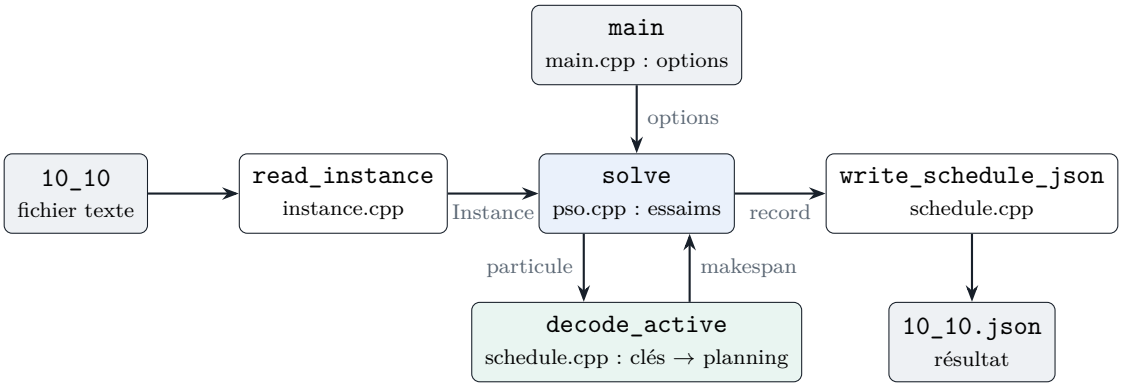


FIGURE 2 – Le trajet des données dans le programme.

TABLE 2 – Les fichiers du projet.

Fichier	Rôle
`src/instance.cpp`	lit le fichier d'instance et vérifie qu'il est bien formé
`src/schedule.cpp`	les deux décodeurs (particule → planning) et l'écriture du JSON
`src/pso.cpp`	l'essaim : mise à jour des particules, topologies, redémarrage, multi-essaims
`src/main.cpp`	lit les options de la ligne de commande et lance le solveur
`tests/test_core.cpp`	tests automatiques (lecture, exemple du cours, PSO)
`tools/validate.py`	vérificateur indépendant d'un JSON produit

3 Lire l'instance (`instance.cpp`)

Le fichier d'entrée a le format des slides : un nom, puis le nombre de jobs et de machines, puis une ligne par job contenant les couples `machine durée`.

```
10_10
10 10
4 88 8 68 6 94 5 99 1 67 2 89 9 77 7 99 0 86 3 92
...
```

En mémoire, l'atelier devient une structure `Instance` : le nom, `jobs`, `machines`, et `operations[job][étape]` qui contient le couple `(machine, duration)`.

La lecture est volontairement **robuste**, car une erreur de lecture le jour du concours coûterait une instance entière :

- les lignes vides, les commentaires `#` et les fins de ligne Windows (`\r`) sont acceptés ;
- si la première ligne contient déjà les dimensions, le nom est pris dans le nom du fichier (le JSON de sortie doit porter le même nom que l'instance) ;
- un fichier mal formé est refusé avec un message clair : couple manquant ou en trop, machine inexistante, machine utilisée deux fois par un même job, job manquant ;
- des lignes en trop après le dernier job déclenchent seulement un avertissement.

4 Représenter une solution : les clés aléatoires

Le PSO a été inventé pour des problèmes *continus* : une particule y est un point de $\mathbb{R}^D$ qui se déplace avec une vitesse. Or un ordonnancement est *discret*. Le pont entre les deux mondes est le codage par **clés aléatoires**.

Idée clé

Une particule est une liste de $D = n \times m$ nombres réels : **une clé par opération**. La clé de l'étape o du job j est rangée à l'indice $j \cdot m + o$. Un *décodeur* transforme ces nombres en planning, ce qui donne un makespan.

Sur l'atelier 3×3, une particule contient donc 9 nombres. Avec le décodeur le plus simple (« semi-actif », section 5.1), on trie les clés par ordre croissant ; chaque clé triée désigne un job, et on obtient une séquence de placement.

TABLE 3 – Une particule qui code l'ordre A. Trier les clés redonne la séquence $J_1, J_0, J_2, J_2, J_0, J_1, J_1, J_2, J_0$, donc le planning de makespan 11.

Opération	J0 · 1	J0 · 2	J0 · 3	J1 · 1	J1 · 2	J1 · 3	J2 · 1	J2 · 2	J2 · 3
Indice $j \cdot m + o$	0	1	2	3	4	5	6	7	8
Clé	0,12	0,40	0,88	0,05	0,52	0,61	0,21	0,33	0,77
Rang après tri	2	5	9	1	6	7	3	4	8

L'intérêt est double : *n'importe quelle* liste de nombres donne un planning valide (les contraintes sont garanties par le décodeur), et l'on peut appliquer telles quelles les formules continues du PSO.

5 Décoder une particule (`schedule.cpp`)

5.1 Le décodeur semi-actif

C'est le décodeur utilisé dans les exemples des slides : on prend la séquence de jobs obtenue par le tri, et on place chaque opération *au plus tôt* derrière celles déjà placées. C'est exactement le calcul début = max(job prêt, machine libre) de la figure 1.

decode_semi_active

```
job_prêt[j] ← 0 ; machine_libre[k] ← 0 ; suivante[j] ← 0
pour chaque job j dans la séquence faire
    (k, durée) ← operations[j][suivante[j]] ; suivante[j] ← suivante[j] + 1
    début ← max(job_prêt[j], machine_libre[k]) ; fin ← début + durée
    job_prêt[j] ← fin ; machine_libre[k] ← fin
retourner le plus grand fin   ▷ le makespan
```

Son défaut : il ne remplit jamais un « trou » laissé sur une machine. Beaucoup de particules différentes donnent de mauvais plannings, et l'essaim perd son temps.

5.2 Le décodeur Giffler-Thompson paramétré (par défaut)

Le décodeur par défaut (`decode_active`) suit l'algorithme de Giffler et Thompson, avec un paramètre δ (Gonçalves *et al.*). Il construit le planning *chronologiquement* et ne consulte la particule qu'au moment d'un **conflit**. À chaque étape :

1. pour chaque job, on calcule le début au plus tôt de sa prochaine opération et sa fin ;
2. φ = la plus petite fin possible. Sa machine M^* est celle où se joue le prochain conflit ;
3. σ = le plus petit début possible sur M^* ;
4. les **candidats** sont les opérations sur M^* qui peuvent commencer dans la fenêtre $[\sigma,\ \sigma + \delta(\varphi - \sigma)]$;
5. **la particule décide** : le candidat dont la clé est la plus petite passe.

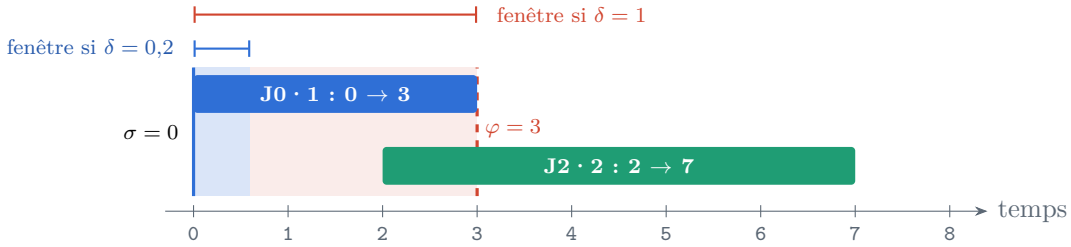


FIGURE 3 – Étape 2 du décodage de l'atelier 3×3 : J0 et J2 veulent tous deux la machine M0. La fenêtre réglée par δ fixe qui est candidat. Avec $\delta = 1$, les deux sont candidats et c'est la clé de la particule qui tranche ; avec $\delta = 0{,}2$, seul J0 peut commencer assez tôt (J2 ne commence qu'à $2 > 0{,}6$).

Le paramètre δ règle la taille de l'espace de recherche. Avec $\delta = 1$, on obtient des plannings *actifs* : on ne peut avancer aucune opération sans en retarder une autre, et l'optimum fait toujours partie de cet ensemble. Avec $\delta = 0$, on obtient des plannings *sans retard* : une machine n'attend jamais quand une opération est prête. Cet ensemble est plus petit et souvent très bon, mais il ne contient pas toujours l'optimum. La valeur retenue, $\delta = 0{,}2$, est un compromis mesuré expérimentalement (section 9).

Sur ce petit exemple, toutes les fenêtres ne contiennent qu'un seul candidat : le décodeur seul construit directement le planning optimal (makespan 11), le même que l'ordre A de la figure 1. Sur une vraie instance 10×10, les conflits à plusieurs candidats sont nombreux, et ce sont eux que l'essaim apprend à trancher.

Vitesse. Le décodeur est appelé des millions de fois. Ses tableaux de travail sont réutilisés d'un appel à l'autre (`thread_local`) et il ne fait que deux passes sur les jobs par opération placée. Il ne construit le planning complet (`schedule_from_starts`) que lorsqu'un nouveau record est trouvé.

TABLE 4 – Déroulé complet du décodeur ($\delta = 0{,}2$) sur l'atelier 3×3. Chaque case de candidat donne la machine et l'intervalle début–fin au plus tôt.

Pas	J0	J1	J2	φ (M^*)	σ	fenêtre	placé
1	M0 0–3	M1 0–4	M2 0–2	2 (M2)	0	[0; 0,4]	J2 : 0–2
2	M0 0–3	M1 0–4	M0 2–7	3 (M0)	0	[0; 0,6]	J0 : 0–3
3	M1 3–5	M1 0–4	M0 3–8	4 (M1)	0	[0; 0,8]	J1 : 0–4
4	M1 4–6	M2 4–7	M0 3–8	6 (M1)	4	[4; 4,4]	J0 : 4–6
5	M2 6–10	M2 4–7	M0 3–8	7 (M2)	4	[4; 4,6]	J1 : 4–7
6	M2 7–11	M0 7–9	M0 3–8	8 (M0)	3	[3; 4]	J2 : 3–8
7	M2 7–11	M0 8–10	M1 8–11	10 (M0)	8	[8; 8,4]	J1 : 8–10
8	M2 7–11	–	M1 8–11	11 (M2)	7	[7; 7,8]	J0 : 7–11
9	–	–	M1 8–11	11 (M1)	8	[8; 8,6]	J2 : 8–11

Résultat : environ 170 000 évaluations par seconde sur un 10×10, et 46 μs par évaluation sur un 30×20.

6 L'essaim particulaire (`pso.cpp`)

6.1 Le principe

L'optimisation par essaims particulaires (Kennedy et Eberhart, 1995) imite une nuée d'oiseaux. Chaque particule i possède :

- une **position** x_i (ses D clés, donc un planning) ;
- une **vitesse** v_i ;
- son **meilleur souvenir** p_i (*pbest*), la meilleure position qu'elle a visitée ;
- un **guide social** g_i : le meilleur souvenir de ses voisins (*gbest* quand le voisinage est tout l'essaim).

À chaque tour, chaque dimension d de chaque particule est mise à jour :

$$v_{i,d} \leftarrow \underbrace{w\, v_{i,d}}_{\text{inertie}} + \underbrace{c_1 r_1 \,(p_{i,d} - x_{i,d})}_{\text{mémoire}} + \underbrace{c_2 r_2 \,(g_{i,d} - x_{i,d})}_{\text{influence sociale}}, \qquad x_{i,d} \leftarrow x_{i,d} + v_{i,d}$$

où r_1, r_2 sont tirés au hasard dans $[0, 1]$ à chaque fois.

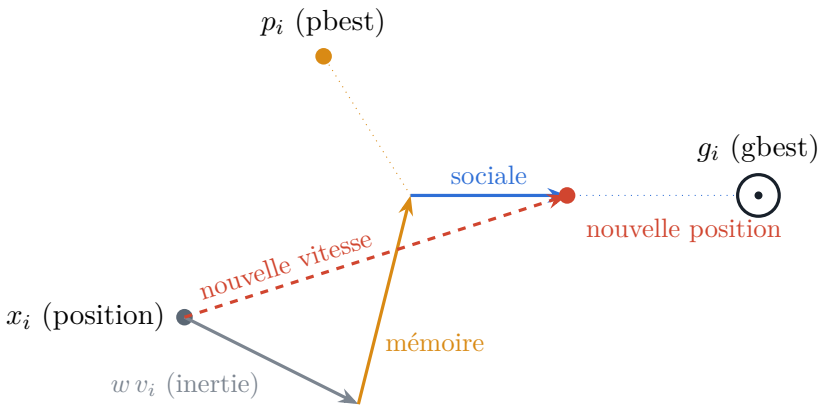


FIGURE 4 – La nouvelle vitesse est la somme de trois tendances : continuer sur sa lancée, revenir vers son meilleur souvenir, et aller vers le meilleur de ses voisins.

6.2 Les réglages retenus

- **Coefficients** $c_1 = c_2 = 1{,}49618$: les valeurs de « constriction » de Clerc et Kennedy, qui garantissent un essaim stable.

— **Inertie décroissante** de $w = 0{,}9$ à $w = 0{,}4$ au fil du temps : l'essaim explore au début et affine à la fin (`--no-decreasing-inertia` fixe $w = 0{,}7298$).
— **Vitesse bornée** à $\pm 0{,}2$ par dimension (`--vmax`) : une particule ne saute pas d'un bout à l'autre de l'espace en un tour.
— **Positions non bornées** : seul l'ordre relatif des clés compte. Si on bloquait les clés dans $[0, 1]$, beaucoup s'empileraient à 0 ou à 1, deviendraient égales, et la particule perdrait de l'information.
— **Plateaux acceptés** : une position de *même* makespan remplace le meilleur souvenir. Le JSP a de larges plateaux de solutions équivalentes ; sans cela, l'essaim s'y figerait.
— **Départ structuré** : dans le premier essaim, trois particules partent de règles de priorité classiques (SPT : opération la plus courte ; MWKR : job avec le plus de travail restant ; MOPNR : job avec le plus d'opérations restantes). Les autres partent au hasard.

6.3 La boucle principale

Un essaim (un thread)

```
initialiser 30 particules (3 règles de priorité + hasard) ; évaluer chacune
tant que le temps n'est pas écoulé faire
    w ← 0,9 - 0,5 × (temps écoulé / budget)
    calculer le guide de chaque particule (voisinage en anneau ou CLPSO)
    pour chaque particule i faire
        mettre à jour v_i et x_i, dimension par dimension
        f ← makespan(décoder(x_i))
        si f ≤ f(p_i) alors p_i ← x_i ; publier si c'est un record global
    si pas de progrès depuis 200 tours alors relancer l'essaim (sauf la meilleure)
    tous les 50 tours : la pire particule adopte le record global (migration)
```

7 Empêcher l'essaim de stagner

Un PSO classique converge **trop vite** : toutes les particules s'agglutinent autour du premier bon planning trouvé, leurs vitesses tombent à zéro et plus rien ne bouge. Sur abz5, la version de départ ne progressait plus après une seconde, sur les 55 disponibles. Quatre mécanismes combattent ce phénomène. Ils restent tous des mécanismes d'essaim.

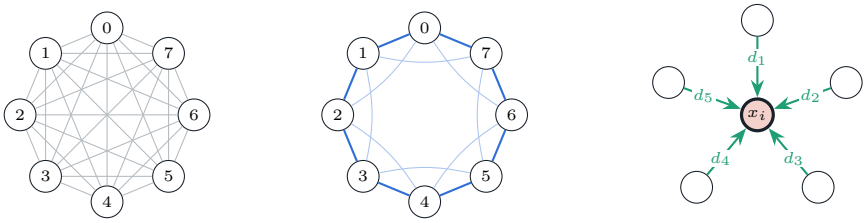


FIGURE 5 – Les trois façons de choisir le guide social d'une particule.

7.1 La topologie en anneau

Les particules sont disposées en cercle. Le guide social de chacune est le meilleur souvenir parmi elle-même et ses $k = 2$ voisins de chaque côté (`--neighbors`). Un bon planning se propage donc de proche en proche au lieu de contaminer tout l'essaim d'un coup : plusieurs zones de l'espace restent explorées en parallèle plus longtemps.

7.2 Le redémarrage partiel

Si le meilleur de l'essaim ne s'améliore plus pendant 200 tours (`--restart`), l'essaim a convergé. On garde sa meilleure particule et on relance toutes les autres au hasard, avec de nouvelles vitesses. Elles repartent explorer d'autres zones, et le meilleur planning n'est jamais perdu.

Nous avons aussi essayé de relancer les particules *autour* de la meilleure (position du record plus un bruit gaussien). C'était moins bon : l'essaim a besoin de vraie diversité, pas d'une variation sur le même thème.

7.3 CLPSO : l'apprentissage global

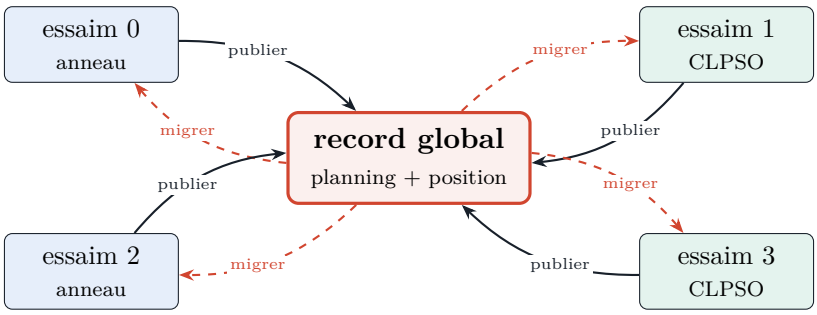
Dans le PSO à apprentissage global (*Comprehensive Learning PSO*, Liang *et al.*, 2006), une particule n'a plus de guide unique. Pour **chaque dimension** d, elle choisit un « exemple » : avec une probabilité $P_c(i)$, le meilleur souvenir d'une autre particule tirée par tournoi (la meilleure de deux tirées au hasard), sinon le sien.

$$v_{i,d} \leftarrow w\, v_{i,d} + c\, r\, (p_{e_i(d),\,d} - x_{i,d}), \qquad P_c(i) = 0{,}05 + 0{,}45\,\frac{e^{10\,i/(N-1)} - 1}{e^{10} - 1}, \qquad c = 1{,}49445$$

Les exemples sont tirés à nouveau quand la particule ne progresse plus pendant 7 tours. Comme chaque particule mélange des morceaux de plusieurs autres, l'essaim ne converge jamais d'un bloc. C'est plus lent, mais cela explore beaucoup mieux. CLPSO entretient lui-même sa diversité : on ne le redémarre pas.

7.4 Le multi-essaims hétérogène

Le programme lance **un essaim par cœur** du processeur (`--threads 0`). Avec la topologie `mixed`, un essaim sur deux tourne en anneau avec redémarrage, les autres en CLPSO. Les mesures de la section 9 montrent que CLPSO est meilleur jusqu'à 15×15 et l'anneau au-delà. Le mélange est donc robuste quelle que soit la taille des instances du concours.



… jusqu'à un essaim par cœur (12 sur la machine de test)

FIGURE 6 – Les essaims tournent en parallèle et ne partagent que le record. Toutes les 50 itérations (`--migrate`), la pire particule de chaque essaim adopte la position du record global.

8 Parallélisme et fiabilité

— **Partage du record.** Le makespan du record est un `std::atomic`, lu sans verrou à chaque évaluation. Le planning et la position du record sont protégés par un `std::mutex`, pris seulement quand un essaim bat le record. Les threads ne se ralentissent donc presque jamais.

— **Hasard reproductible par essaim.** Chaque thread a son propre générateur (`mt19937_64`), initialisé à partir de `--seed` et du numéro du thread.

— **Budget de temps.** Le temps est mesuré avec une horloge monotone (`steady_clock`), insensible aux changements d'heure du système : 55 secondes par défaut pour une limite d'une minute.
— **Rien n'est perdu.** Le meilleur planning est écrit dans le JSON au plus toutes les 2 secondes pendant la recherche, puis une dernière fois à la fin. Si le programme est interrompu, le dernier record est sur le disque.
— **Vérification.** `tools/validate.py` relit l'instance et le JSON, et contrôle les trois contraintes et le makespan, indépendamment du code C++.

Le format de sortie. Pour l'atelier 3×3 avec l'ordre A :

```
{
  "makespan": 11,
  "jobs_schedule": {
    "Job_0": [{"machine": 0, "start": 0, "end": 3},
              {"machine": 1, "start": 4, "end": 6},
              {"machine": 2, "start": 7, "end": 11}],
    "Job_1": [{"machine": 1, "start": 0, "end": 4},
              {"machine": 2, "start": 4, "end": 7},
              {"machine": 0, "start": 8, "end": 10}],
    "Job_2": [{"machine": 2, "start": 0, "end": 2},
              {"machine": 0, "start": 3, "end": 8},
              {"machine": 1, "start": 8, "end": 11}]
  }
}
```

9 Résultats

Les mesures portent sur 7 instances de la bibliothèque JSPLIB, de 10×10 à 30×20, dont l'optimum (ou la meilleure solution connue) est publié. Le critère est l'**écart** au meilleur connu :

$$\text{écart} = \frac{\text{makespan obtenu} - \text{meilleur connu}}{\text{meilleur connu}}.$$

9.1 Convergence sur l'instance des slides

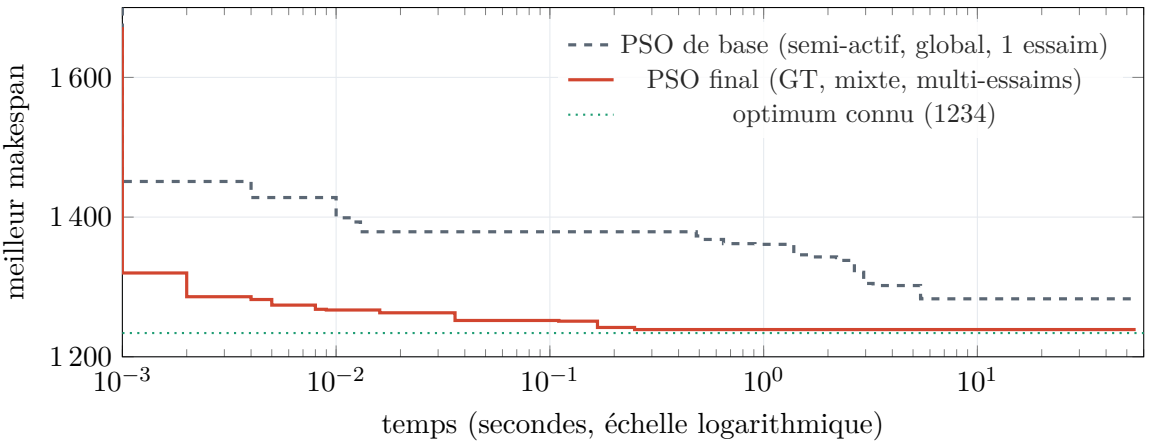


FIGURE 7 – Évolution du meilleur makespan sur abz5 (l'instance `10_10` des slides), en 55 secondes. Chaque marche est un nouveau record. Le PSO de base trouve son dernier record (1283, soit 4,0 % au-dessus de l'optimum) après 5,4 s puis n'avance plus. Le PSO final atteint 1239 (0,4 %) dès 0,25 s : sur une instance de cette taille, ses 12 essaims trouvent très vite un planning proche de l'optimum, puis plafonnent eux aussi.

9.2 Ce qu'apporte chaque mécanisme

Chaque ligne ajoute un mécanisme à la précédente. Les mesures sont faites en 10 secondes sur 2 à 3 graines aléatoires, ce qui permet de comparer rapidement. La dernière ligne est mesurée sur le vrai budget de 55 secondes.

Table 5 – Écart moyen au meilleur connu sur les 7 instances.

Configuration	Écart moyen
Décodeur semi-actif, topologie globale (version de départ)	22,3 %
Ancien décodeur « GT » approximatif	10,4 %
Nouveau décodeur Giffler-Thompson paramétré	6,8 %
+ topologie en anneau	6,4 %
+ redémarrage après 200 tours sans progrès	5,9 %
+ voisinage de 2 et $\delta = 0{,}2$	5,1 %
+ 12 essaims indépendants (un par cœur)	4,7 %
+ migration du record toutes les 50 itérations	4,2 %
+ essaims mixtes anneau / CLPSO	3,8 %
Configuration finale, 55 secondes	**3,3 %**

9.3 Résultats par instance

Table 6 – Version de départ (10 s, 1 essaim) contre version finale (55 s, 12 essaims).

Instance	Taille	Meilleur connu	Départ	Final	Makespan final
abz5 (`10_10`)	10×10	1234	3,8 %	0,4 %	1239
ft10	10×10	930	7,5 %	2,0 %	949
la21	15×10	1046	9,4 %	0,9 %	1055
la26	20×10	1218	8,4 %	**0,0 %**	**1218**
ta01	15×15	1231	10,4 %	4,1 %	1282
ta21	20×20	1644	14,7 %	5,2 %	1729
ta41	30×20	2018	18,5 %	10,7 %	2234
Moyenne			**10,4 %**	**3,3 %**	

Table 7 – Anneau contre CLPSO seuls (55 s, 1 essaim, moyenne de 3 graines) : des profils complémentaires, d'où le choix d'essaims mixtes.

	abz5	ft10	la21	la26	ta01	ta21	ta41
Anneau + redémarrage	0,41	2,15	2,96	0,93	4,93	**6,97**	**12,09**
CLPSO	0,49	2,15	**1,15**	**0,00**	**4,01**	7,89	14,87

9.4 Ce que nous avons appris

— **Le décodeur compte autant que l'essaim.** Changer de décodeur a fait plus que tous les réglages d'essaim réunis : c'est lui qui définit l'espace où les particules cherchent.

— **La stagnation se combat par la diversité, pas par le temps.** Doubler le temps de calcul n'a presque rien changé (5,17 % → 5,09 %), alors que les topologies, les redémarrages et les essaims multiples ont fait gagner plusieurs points.

— **Il faut mesurer avant de garder une idée.** La relance autour du record semblait prometteuse ; elle faisait moins bien qu'une relance aléatoire. Les petits réglages (nombre de particules, inertie, `vmax`) donnaient des écarts plus petits que le bruit de mesure (±0,3 %).
— **Limite.** Les très grandes instances restent difficiles (ta41 à 10,7 %) : avec 600 clés par particule, un PSO pur peine à explorer un espace aussi vaste en une minute.

10 Utilisation

Compiler (une fois, en mode optimisé) :

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
```

Lancer une instance avec les réglages par défaut, qui sont les meilleurs trouvés :

```
./build/jsp_pso instances/abz5 --out resultats
python3 tools/validate.py instances/abz5 resultats/abz5.json
```

Table 8 – Les options de la ligne de commande.

Option	Défaut	Rôle
`--time S`	55	budget en secondes (0 = sans limite)
`--threads N`	0	nombre d'essaims, 0 = un par cœur
`--topology`	mixed	`ring`, `global`, `cl` (CLPSO) ou `mixed`
`--neighbors K`	2	voisins de chaque côté dans l'anneau
`--restart N`	200	tours sans progrès avant relance (0 = jamais)
`--migrate N`	50	période de migration du record (0 = jamais)
`--particles N`	30	particules par essaim
`--vmax V`	0,2	vitesse maximale par dimension
`--decoder`	gt	`gt` (Giffler-Thompson) ou `semi` (semi-actif)
`--delta D`	0,2	fenêtre du décodeur `gt`
`--seed N`	0	graine du hasard
`--out DIR`	.	dossier du JSON de sortie
`--target M`		s'arrêter dès qu'un makespan $\leq M$ est trouvé
`--log FICHIER`		écrire la courbe de convergence (CSV)

Tests automatiques. `ctest` vérifie la lecture de l'exemple du cours et de huit fichiers volontairement défectueux, le décodage de l'ordre A (makespan 11), l'écriture du JSON, et que le PSO retrouve l'optimum 11 sur l'atelier 3×3. Une compilation avec *sanitizers* (`-DJSP_SANITIZE=ON`) détecte en plus les erreurs mémoire et les comportements indéfinis.